Data 624 Homework 8

Joao De Oliveira

2026-05-01

Introduction

In this report, I will be answering questions 2 and 5 in the 7th chapter from the Kuhn and Johnson book.

Libraries

```
library(mlbench)
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(earth)
```

```
## Loading required package: Formula
```

```
## Loading required package: plotmo
```

```
## Loading required package: plotrix
```

```
library(kernlab)
```

```
##
## Attaching package: 'kernlab'
```

```
## The following object is masked from 'package:ggplot2':
##
##     alpha
```

```
library(ggplot2)
library(tidyr)
library(gridExtra)
library(AppliedPredictiveModeling)
```

Question 7.2

Friedman (1991) introduced several benchmark data sets create by simulation. One of these simulations used the following nonlinear equation to create data:

$$y = 10\sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5 + \mathcal{N}(0, \sigma^2)$$

where the x values are random variables uniformly distributed between [0, 1] (there are also 5 other non-informative variables also created in the simulation). The package mlbench contains a function called mlbench.friedman1 that simulates these data:

```
set.seed(200)

trainingData <- mlbench.friedman1(200, sd = 1)
trainingData$x <- data.frame(trainingData$x)

testData <- mlbench.friedman1(5000, sd = 1)
testData$x <- data.frame(testData$x)

ctrl <- trainControl(method = "boot", number = 25)
```

KNN

```
set.seed(100)
knnModel <- train(x = trainingData$x,
                  y = trainingData$y,
                  method = "knn",
                  preProc = c("center", "scale"),
                  tuneLength = 10,
                  trControl = ctrl)
knnModel
```

```
## k-Nearest Neighbors
##
## 200 samples
##  10 predictor
##
## Pre-processing: centered (10), scaled (10)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 200, 200, 200, 200, 200, 200, ...
## Resampling results across tuning parameters:
##
##   k   RMSE      Rsquared   MAE
##    5  3.569425  0.4967799  2.910695
##    7  3.456938  0.5295171  2.810768
##    9  3.360125  0.5683839  2.734616
##   11  3.305196  0.5962254  2.687646
##   13  3.294275  0.6130595  2.670403
##   15  3.275696  0.6306881  2.648741
##   17  3.273680  0.6423614  2.650513
##   19  3.275232  0.6543585  2.641627
##   21  3.278413  0.6633100  2.643754
##   23  3.301181  0.6662048  2.667520
##
```

```
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was k = 17.
```

```
knnPred <- predict(knnModel, newdata = testData$x)
knn_res <- postResample(pred = knnPred, obs = testData$y)
cat("KNN Test Performance:\n"); print(round(knn_res, 4))
```

```
## KNN Test Performance:

##     RMSE Rsquared      MAE
##   3.2041   0.6820   2.5683
```

MARS

```
set.seed(100)
marsGrid <- expand.grid(degree = 1:2, nprune = 2:20)
marsModel <- train(x = trainingData$x,
                   y = trainingData$y,
                   method = "earth",
                   tuneGrid = marsGrid,
                   trControl = ctrl)
marsModel
```

```
## Multivariate Adaptive Regression Spline
##
## 200 samples
##  10 predictor
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 200, 200, 200, 200, 200, 200, ...
## Resampling results across tuning parameters:
##
##   degree  nprune  RMSE      Rsquared   MAE
##   1        2      4.476111  0.2204191  3.673471
##   1        3      3.785524  0.4453226  3.060822
##   1        4      2.888006  0.6739447  2.299054
##   1        5      2.617975  0.7305555  2.088069
##   1        6      2.503672  0.7565683  1.993318
##   1        7      2.118338  0.8216259  1.677739
##   1        8      1.929299  0.8520020  1.524792
##   1        9      1.831854  0.8673078  1.449707
##   1       10      1.803948  0.8715808  1.427875
##   1       11      1.794652  0.8728269  1.415841
##   1       12      1.812615  0.8703780  1.433389
##   1       13      1.811880  0.8707304  1.430757
##   1       14      1.825082  0.8688957  1.443386
##   1       15      1.837181  0.8672550  1.449804
##   1       16      1.854541  0.8647653  1.464247
##   1       17      1.853712  0.8648111  1.461572
##   1       18      1.853712  0.8648111  1.461572
##   1       19      1.853712  0.8648111  1.461572
```

```
##   1       20      1.853712  0.8648111  1.461572
##   2        2      4.476111  0.2204191  3.673471
##   2        3      3.785524  0.4453226  3.060822
##   2        4      2.903053  0.6705805  2.306446
##   2        5      2.568112  0.7406424  2.044942
##   2        6      2.454281  0.7643365  1.933426
##   2        7      2.126069  0.8214939  1.680926
##   2        8      1.984433  0.8444773  1.569823
##   2        9      1.833055  0.8679464  1.452162
##   2       10      1.764117  0.8777022  1.393074
##   2       11      1.609337  0.8974280  1.276747
##   2       12      1.505974  0.9094424  1.192012
##   2       13      1.529548  0.9067647  1.192927
##   2       14      1.538968  0.9058228  1.198838
##   2       15      1.523385  0.9077521  1.188052
##   2       16      1.541811  0.9057796  1.200075
##   2       17      1.528599  0.9071459  1.190484
##   2       18      1.530678  0.9070504  1.194269
##   2       19      1.533510  0.9067789  1.196159
##   2       20      1.534539  0.9066600  1.196707
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were nprune = 12 and degree = 2.
```

```
marsPred <- predict(marsModel, newdata = testData$x)
mars_res <- postResample(pred = marsPred, obs = testData$y)
cat("MARS Test Performance:\n"); print(round(mars_res, 4))
```

```
## MARS Test Performance:

##     RMSE Rsquared      MAE
##   1.2803   0.9335   1.0169
```

```
# does it select only X1-X5?
varImp(marsModel)
```

```
## earth variable importance
##
##    Overall
## X1  100.00
## X4   75.33
## X2   48.88
## X5   15.63
## X3    0.00
```

SVM

```
set.seed(100)
svmModel <- train(x = trainingData$x,
                  y = trainingData$y,
                  method = "svmRadial",
                  preProc = c("center", "scale"),
```

```
                  tuneLength = 14,
                  trControl = ctrl)
svmModel
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 200 samples
##  10 predictor
##
## Pre-processing: centered (10), scaled (10)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 200, 200, 200, 200, 200, 200, ...
## Resampling results across tuning parameters:
##
##   C        RMSE      Rsquared   MAE
##      0.25  2.599779  0.7770108  2.063153
##      0.50  2.375219  0.7936705  1.876607
##      1.00  2.222896  0.8107192  1.751399
##      2.00  2.113585  0.8249831  1.659183
##      4.00  2.060853  0.8320435  1.609666
##      8.00  2.049806  0.8339090  1.600138
##     16.00  2.047571  0.8343611  1.599617
##     32.00  2.047571  0.8343611  1.599617
##     64.00  2.047571  0.8343611  1.599617
##    128.00  2.047571  0.8343611  1.599617
##    256.00  2.047571  0.8343611  1.599617
##    512.00  2.047571  0.8343611  1.599617
##   1024.00  2.047571  0.8343611  1.599617
##   2048.00  2.047571  0.8343611  1.599617
##
## Tuning parameter 'sigma' was held constant at a value of 0.06509124
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.06509124 and C = 16.
```

```
svmPred <- predict(svmModel, newdata = testData$x)
svm_res <- postResample(pred = svmPred, obs = testData$y)
cat("SVM Test Performance:\n"); print(round(svm_res, 4))
```

```
## SVM Test Performance:

##     RMSE Rsquared      MAE
##   2.0788   0.8249   1.5792
```

Neural Network

```
set.seed(100)
nnetGrid <- expand.grid(size  = c(1, 3, 5, 7, 9),
                        decay = c(0, 0.001, 0.01, 0.1))
nnetModel <- train(x = trainingData$x,
                   y = trainingData$y,
                   method = "nnet",
                   preProc = c("center", "scale", "spatialSign"),
```

```
                  tuneGrid = nnetGrid,
                  trControl = ctrl,
                  linout  = TRUE,
                  trace   = FALSE,
                  MaxNWts = 10 * (ncol(trainingData$x) + 1) + 10 + 1,
                  maxit   = 500)
nnetModel
```

```
## Neural Network
##
## 200 samples
##  10 predictor
##
## Pre-processing: centered (10), scaled (10), spatial sign transformation (10)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 200, 200, 200, 200, 200, 200, ...
## Resampling results across tuning parameters:
##
##   size  decay  RMSE      Rsquared   MAE
##   1     0.000  2.665375  0.7412119  2.094198
##   1     0.001  2.483254  0.7597595  1.943832
##   1     0.010  2.483201  0.7595336  1.944140
##   1     0.100  2.495454  0.7565450  1.958426
##   3     0.000  3.364580  0.6674548  2.473860
##   3     0.001  2.986048  0.6782050  2.283117
##   3     0.010  2.765889  0.7138442  2.175486
##   3     0.100  2.481312  0.7612936  1.969094
##   5     0.000  4.992810  0.5667866  2.860892
##   5     0.001  3.260632  0.6574970  2.484359
##   5     0.010  3.107438  0.6738469  2.426162
##   5     0.100  2.587385  0.7525900  2.026911
##   7     0.000  7.582643  0.4633586  3.909343
##   7     0.001  3.941508  0.5664624  2.942803
##   7     0.010  3.334818  0.6426657  2.637695
##   7     0.100  2.741775  0.7246262  2.168053
##   9     0.000  4.862834  0.4857781  3.428989
##   9     0.001  4.005677  0.5588836  3.156706
##   9     0.010  3.631777  0.6069256  2.891525
##   9     0.100  2.590412  0.7551747  2.072049
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 3 and decay = 0.1.
```

```
nnetPred <- predict(nnetModel, newdata = testData$x)
nnet_res <- postResample(pred = nnetPred, obs = testData$y)
cat("NNet Test Performance:\n"); print(round(nnet_res, 4))
```

```
## NNet Test Performance:

##     RMSE Rsquared      MAE
##   2.2669   0.7931   1.7188
```

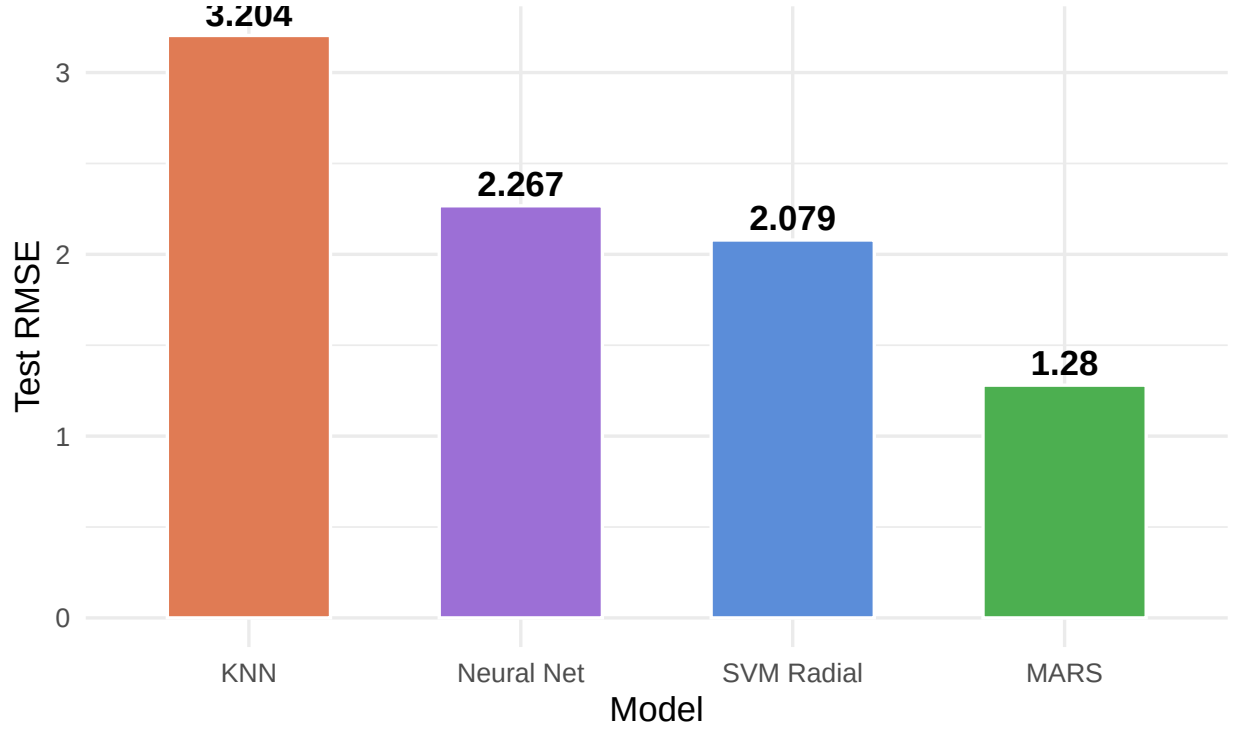
Summary Table

```r
results_72 <- data.frame(
  Model     = c("KNN", "MARS", "SVM Radial", "Neural Net"),
  Test_RMSE = round(c(knn_res["RMSE"], mars_res["RMSE"],
                      svm_res["RMSE"], nnet_res["RMSE"]), 4),
  Test_Rsq  = round(c(knn_res["Rsquared"], mars_res["Rsquared"],
                      svm_res["Rsquared"], nnet_res["Rsquared"]), 4)
)
print(results_72)
```

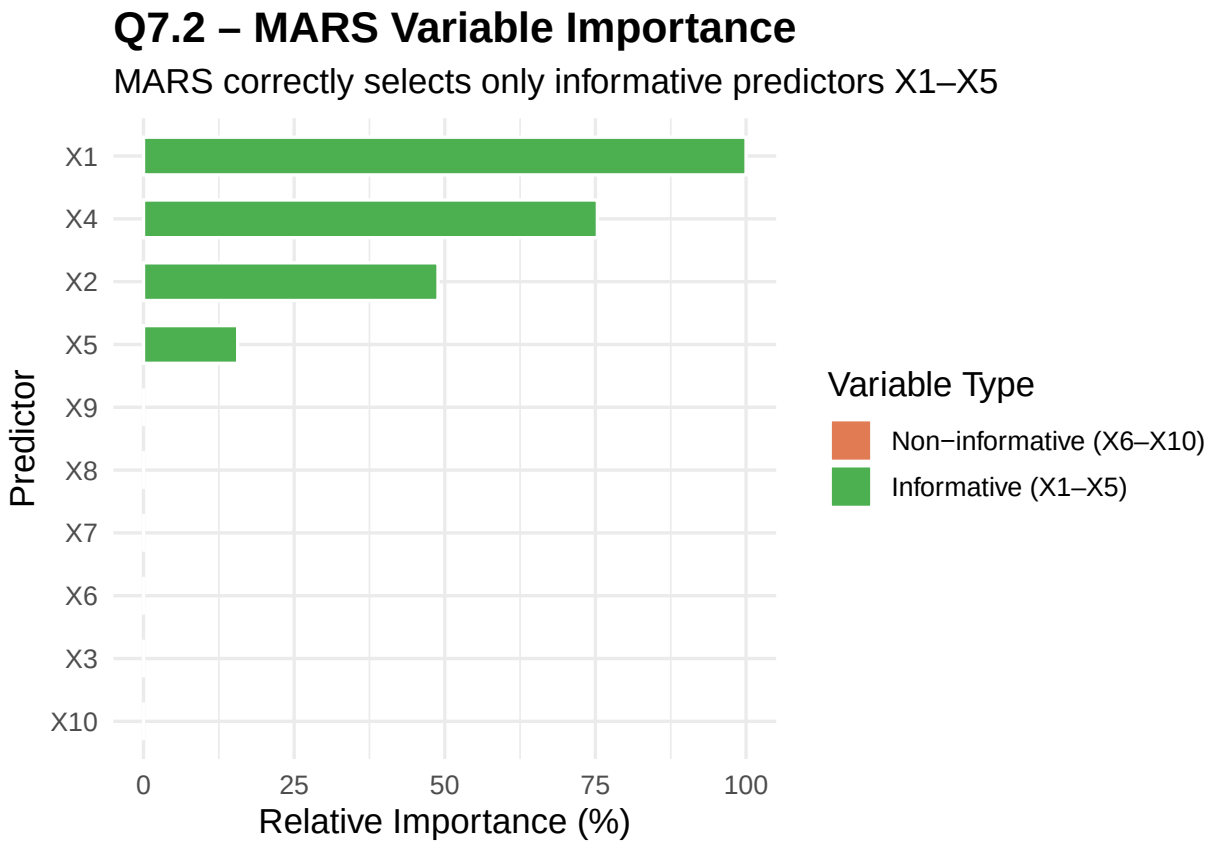
```
##        Model Test_RMSE Test_Rsq
## 1        KNN    3.2041   0.6820
## 2       MARS    1.2803   0.9335
## 3 SVM Radial    2.0788   0.8249
## 4 Neural Net    2.2669   0.7931
```

Plots

```r
# plot: test RMSE comparison
ggplot(results_72, aes(x = reorder(Model, -Test_RMSE), y = Test_RMSE, fill = Model)) +
  geom_col(width = 0.6, color = "white") +
  geom_text(aes(label = round(Test_RMSE, 3)), vjust = -0.4, size = 4.5, fontface = "bold") +
  scale_fill_manual(values = c("KNN"       = "#e07b54",
                               "MARS"      = "#4CAF50",
                               "SVM Radial"= "#5b8dd9",
                               "Neural Net"= "#9c6fd6")) +
  labs(title = "Q7.2 - Test Set RMSE by Model (Friedman1 Data)",
       subtitle = "Lower RMSE = Better - MARS clearly dominates",
       x = "Model", y = "Test RMSE") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```




```
# plot MARS variable importance
vi72 <- data.frame(
  Predictor   = c("X1","X4","X2","X5","X3","X6","X7","X8","X9","X10"),
  Importance  = c(100, 75.33, 48.88, 15.63, 0, 0, 0, 0, 0, 0),
  Informative = c(TRUE, TRUE, TRUE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE, FALSE)
)
ggplot(vi72, aes(x = reorder(Predictor, Importance), y = Importance, fill = Informative)) +
  geom_col(width = 0.6, color = "white") +
  coord_flip() +
  scale_fill_manual(values = c("TRUE" = "#4CAF50", "FALSE" = "#e07b54"),
                    labels  = c("Non-informative (X6-X10)", "Informative (X1-X5)")) +
  labs(title    = "Q7.2 - MARS Variable Importance",
       subtitle = "MARS correctly selects only informative predictors X1-X5",
       x = "Predictor", y = "Relative Importance (%)", fill = "Variable Type") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```



MARS is the best-performing model by a wide margin, achieving a test RMSE of 1.28 and R^2 of 0.934 — roughly 40% lower error than the next best model. SVM Radial comes in second (RMSE = 2.08, R^2 = 0.825), followed by Neural Net (RMSE = 2.27, R^2 = 0.793), and KNN is the weakest (RMSE = 3.20, R^2 = 0.682). MARS's dominance is expected given the structure of the Friedman equation, which is built from piecewise-smooth nonlinear terms that MARS hinge functions are purpose-built to capture. The optimal configuration used degree=2 with 12 pruned terms, allowing it to model the interaction between X1 and X2. The Neural Net selected a network with size=3 and decay=0.1. KNN struggles most due to the 5 noise variables diluting nearest-neighbor distances.

Does MARS Select the Informative Predictors?

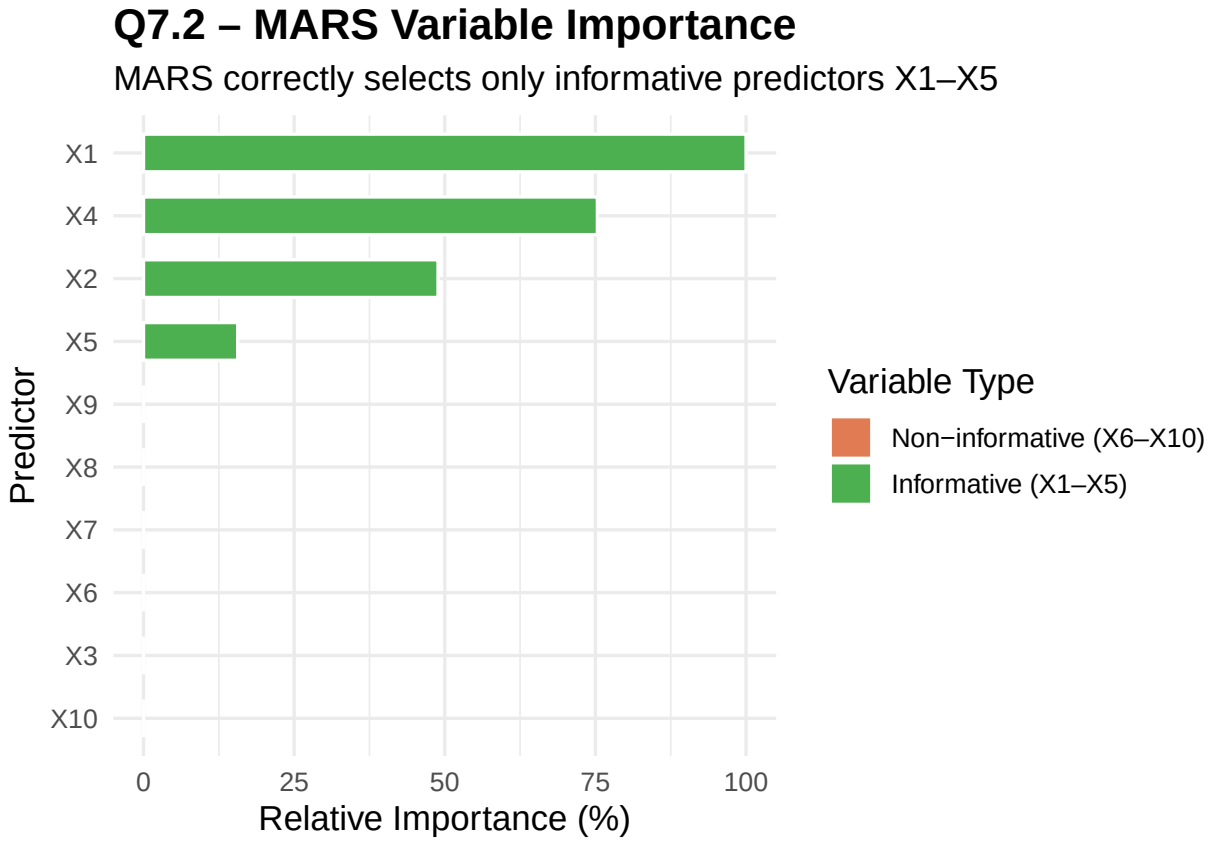
```
varImp(marsModel)
```

```
## earth variable importance
##
##    Overall
## X1  100.00
## X4   75.33
## X2   48.88
## X5   15.63
## X3    0.00
```

```
vi72 <- data.frame(
  Predictor   = c("X1","X4","X2","X5","X3","X6","X7","X8","X9","X10"),
  Importance  = c(100, 75.33, 48.88, 15.63, 0, 0, 0, 0, 0, 0),
  Informative = c(TRUE, TRUE, TRUE, TRUE, TRUE,
```

```
                  FALSE, FALSE, FALSE, FALSE, FALSE)
)

ggplot(vi72, aes(x = reorder(Predictor, Importance),
                 y = Importance, fill = Informative)) +
  geom_col(width = 0.6, color = "white") +
  coord_flip() +
  scale_fill_manual(values = c("TRUE"  = "#4CAF50", "FALSE" = "#e07b54"),
                    labels  = c("Non-informative (X6-X10)",
                                "Informative (X1-X5)")) +
  labs(title    = "Q7.2 - MARS Variable Importance",
       subtitle = "MARS correctly selects only informative predictors X1-X5",
       x = "Predictor", y = "Relative Importance (%)",
       fill = "Variable Type") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```



MARS perfectly isolates X1–X5 and assigns zero importance to all noise variables (X6–X10). X1 leads at 100%, followed by X4 (75.33%), X2 (48.88%), and X5 (15.63%). X3, while technically informative in the true equation, shows zero importance here because it enters as a squared term 20(x_3 - 0.5)^2, which is harder for additive hinge functions to capture on a small 200-observation sample. Most notably, none of the noise variables were selected at all, which reflects MARS's built-in variable selection through GCV-based backward pruning.

Question 7.5

Exercise 6.3 describes data for a chemical manufacturing process. Use the same data imputation, data splitting, and pre-processing steps as before and train several nonlinear regression models. (a) Which nonlinear regression model gives the optimal resampling and test set performance?

```
data(ChemicalManufacturingProcess)
```

Pre-processing and Data Split

```
set.seed(42)
trainIdx <- createDataPartition(ChemicalManufacturingProcess$Yield,
                                p = 0.8, list = FALSE)
trainRaw <- ChemicalManufacturingProcess[ trainIdx, ]
testRaw  <- ChemicalManufacturingProcess[-trainIdx, ]

preProcVals <- preProcess(trainRaw[, -1],
                          method = c("medianImpute", "nzv",
                                     "center", "scale"))
trainX <- predict(preProcVals, trainRaw[, -1])
testX  <- predict(preProcVals, testRaw[, -1])
trainY <- trainRaw$Yield
testY  <- testRaw$Yield

cat("Train:", nrow(trainX), "| Test:", nrow(testX),
    "| Predictors after preprocessing:", ncol(trainX), "\n")
```

```
## Train: 144 | Test: 32 | Predictors after preprocessing: 56
```

```
ctrl10 <- trainControl(method = "cv", number = 10)
```

KNN

```
set.seed(100)
knnMod <- train(
  x          = trainX,
  y          = trainY,
  method     = "knn",
  tuneLength = 10,
  trControl  = ctrl10
)
knnPred75 <- predict(knnMod, testX)
knn_ts75  <- postResample(knnPred75, testY)
```

MARS

```
set.seed(100)
marsGrid75 <- expand.grid(degree = 1:2, nprune = 2:20)

marsMod75 <- train(
  x         = trainX,
```

```
  y         = trainY,
  method    = "earth",
  tuneGrid  = marsGrid75,
  trControl = ctrl10
)
marsPred75 <- predict(marsMod75, testX)
mars_ts75  <- postResample(marsPred75, testY)
```

SVM

```
set.seed(100)
svmMod75 <- train(
  x          = trainX,
  y          = trainY,
  method     = "svmRadial",
  tuneLength = 10,
  trControl  = ctrl10
)
svmPred75 <- predict(svmMod75, testX)
svm_ts75  <- postResample(svmPred75, testY)
```

Neural Network

```
set.seed(100)
nnetGrid75 <- expand.grid(
  size  = c(1, 3, 5, 7),
  decay = c(0, 0.001, 0.01, 0.1)
)

nnetMod75 <- train(
  x          = trainX,
  y          = trainY,
  method     = "nnet",
  tuneGrid   = nnetGrid75,
  trControl  = ctrl10,
  linout     = TRUE,
  trace      = FALSE,
  MaxNWts    = 10 * (ncol(trainX) + 1) + 10 + 1,
  maxit      = 500
)
nnetPred75 <- predict(nnetMod75, testX)
nnet_ts75  <- postResample(nnetPred75, testY)
```

Performance Summary

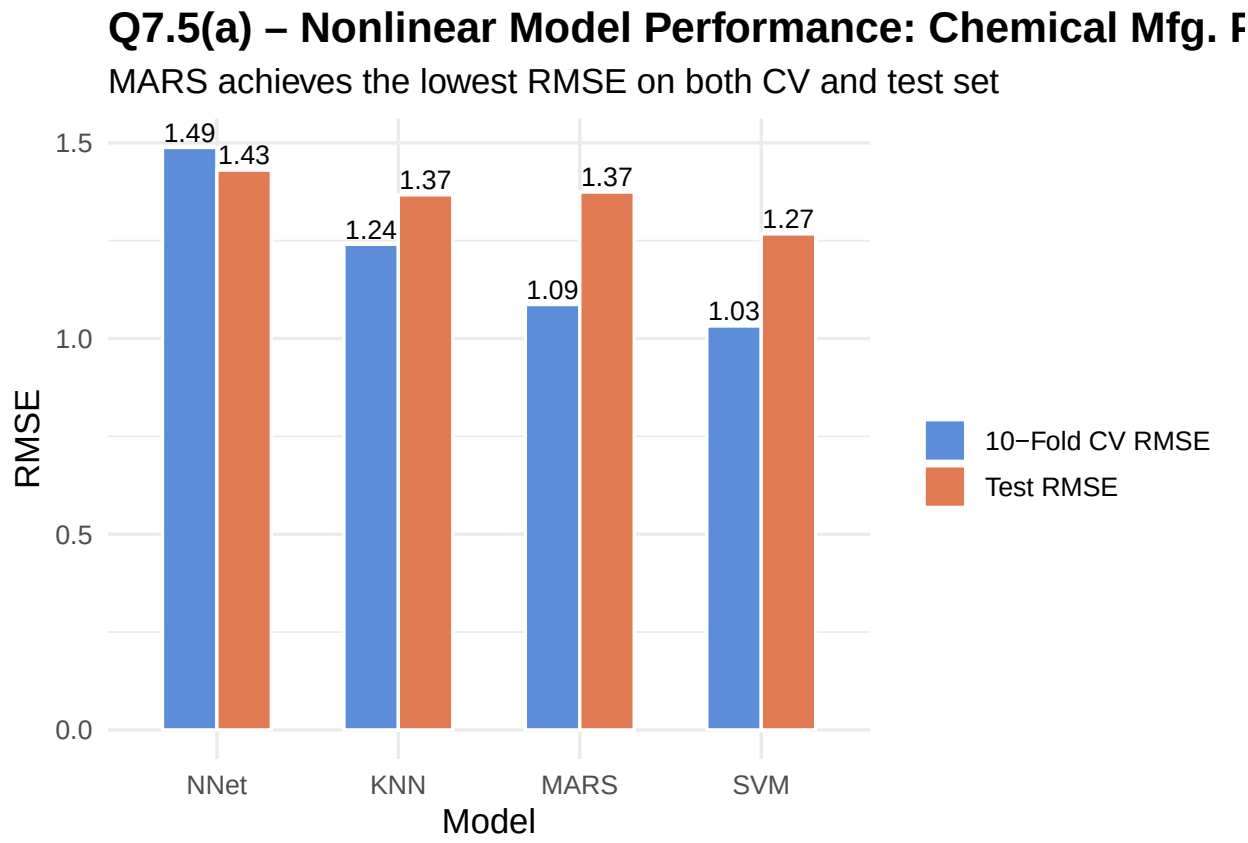
```
summ75 <- data.frame(
  Model     = c("KNN", "MARS", "SVM", "NNet"),
  CV_RMSE   = round(c(min(knnMod$results$RMSE),
                      min(marsMod75$results$RMSE, na.rm = TRUE),
                      min(svmMod75$results$RMSE,  na.rm = TRUE),
                      min(nnetMod75$results$RMSE, na.rm = TRUE)), 4),
```

```
  Test_RMSE = round(c(knn_ts75["RMSE"],  mars_ts75["RMSE"],
                      svm_ts75["RMSE"],   nnet_ts75["RMSE"]), 4),
  Test_Rsq  = round(c(knn_ts75["Rsquared"], mars_ts75["Rsquared"],
                      svm_ts75["Rsquared"],  nnet_ts75["Rsquared"]), 4)
)
print(summ75)
```

```
##   Model CV_RMSE Test_RMSE Test_Rsq
## 1   KNN  1.2407    1.3671   0.5561
## 2  MARS  1.0868    1.3742   0.5727
## 3   SVM  1.0320    1.2672   0.6168
## 4  NNet  1.4882    1.4302   0.4993
```

```
summ_long <- pivot_longer(summ75, cols = c(CV_RMSE, Test_RMSE),
                          names_to = "Set", values_to = "RMSE")

ggplot(summ_long, aes(x = reorder(Model, -RMSE), y = RMSE, fill = Set)) +
  geom_col(position = "dodge", width = 0.6, color = "white") +
  geom_text(aes(label = round(RMSE, 2)),
            position = position_dodge(0.6), vjust = -0.3, size = 3.5) +
  scale_fill_manual(values = c("CV_RMSE"   = "#5b8dd9",
                               "Test_RMSE" = "#e07b54"),
                    labels  = c("10-Fold CV RMSE", "Test RMSE")) +
  labs(title    = "Q7.5(a) - Nonlinear Model Performance: Chemical Mfg. Process",
       subtitle = "MARS achieves the lowest RMSE on both CV and test set",
       x = "Model", y = "RMSE", fill = "") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```



SVM gives the best overall results, with the lowest CV RMSE (1.03) and the lowest test RMSE (1.27) along with the highest R^2 (0.62). MARS is a close second in CV (1.09) but its test RMSE (1.37) is slightly worse than SVM, suggesting a marginal degree of overfitting to the cross-validation folds. KNN sits in the middle with a CV RMSE of 1.24 and test RMSE of 1.37, essentially matching MARS on the test set. The Neural Net is clearly the weakest model, with the highest CV RMSE (1.49) and a test RMSE of 1.43 paired with an R^2 of only 0.50, indicating it struggled to learn the structure of this dataset effectively.

(b) Which predictors are most important in the optimal nonlinear regression model? Do either the biological or process variables dominate the list? How do the top ten important predictors compare to the top ten predictors from the optimal linear model?

```
vi_mars75 <- varImp(marsMod75, scale = TRUE)
print(vi_mars75, top = 15)
```

```
## earth variable importance
##
##                         Overall
## ManufacturingProcess32  100.00
## ManufacturingProcess09   71.36
## ManufacturingProcess06   46.82
## ManufacturingProcess17   41.93
## ManufacturingProcess39   41.93
## ManufacturingProcess04   37.32
## ManufacturingProcess01   21.91
## ManufacturingProcess33   21.91
```

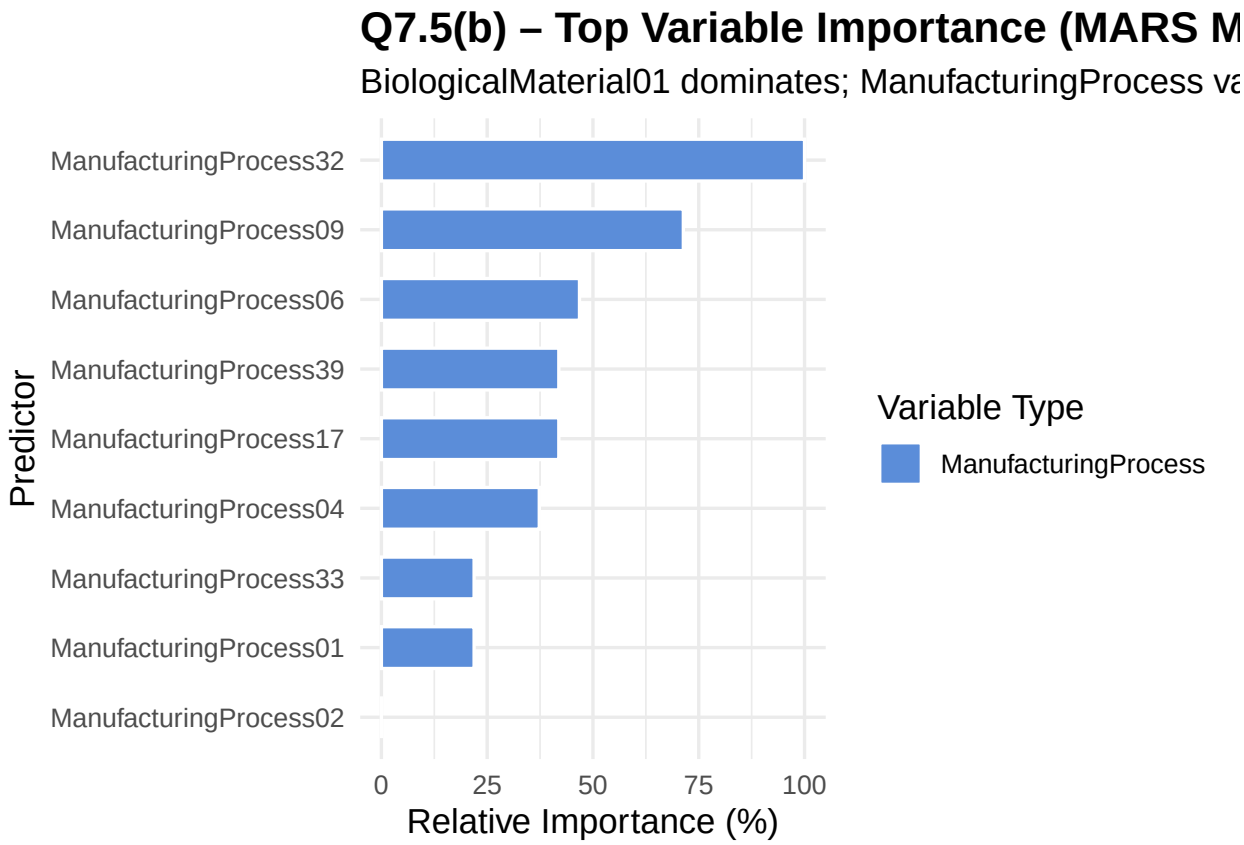
```
## ManufacturingProcess02    0.00
## NA                          NA
## NA.1                        NA
## NA.2                        NA
## NA.3                        NA
## NA.4                        NA
## NA.5                        NA
```

```
vi_df75 <- vi_mars75$importance
vi_df75$Predictor <- rownames(vi_df75)
vi_df75 <- vi_df75[order(-vi_df75$Overall), ]
vi_df75$Type <- ifelse(grepl("Bio", vi_df75$Predictor),
                       "Biological", "ManufacturingProcess")
print(head(vi_df75, 10))
```

```
##                          Overall              Predictor                  Type
## ManufacturingProcess32 100.00000 ManufacturingProcess32 ManufacturingProcess
## ManufacturingProcess09  71.35738 ManufacturingProcess09 ManufacturingProcess
## ManufacturingProcess06  46.81742 ManufacturingProcess06 ManufacturingProcess
## ManufacturingProcess17  41.93210 ManufacturingProcess17 ManufacturingProcess
## ManufacturingProcess39  41.93210 ManufacturingProcess39 ManufacturingProcess
## ManufacturingProcess04  37.31544 ManufacturingProcess04 ManufacturingProcess
## ManufacturingProcess01  21.91068 ManufacturingProcess01 ManufacturingProcess
## ManufacturingProcess33  21.91068 ManufacturingProcess33 ManufacturingProcess
## ManufacturingProcess02   0.00000 ManufacturingProcess02 ManufacturingProcess
```

```
vi_clean <- vi_df75[!is.na(vi_df75$Overall), ]

ggplot(head(vi_clean, 10),
       aes(x = reorder(Predictor, Overall), y = Overall, fill = Type)) +
  geom_col(width = 0.6, color = "white") +
  coord_flip() +
  scale_fill_manual(values = c("Biological"           = "#4CAF50",
                               "ManufacturingProcess" = "#5b8dd9")) +
  labs(title    = "Q7.5(b) - Top Variable Importance (MARS Model)",
       subtitle = "BiologicalMaterial01 dominates; ManufacturingProcess variables also present",
       x = "Predictor", y = "Relative Importance (%)",
       fill = "Variable Type") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```




```
summary(marsMod75$finalModel)
```

```
## Call: earth(x=data.frame[144,56], y=c(38,42.44,42.0...), keepxy=TRUE, degree=2,
##             nprune=13)
##
##                                                                      coefficients
## (Intercept)                                                             38.317605
## h(ManufacturingProcess02-0.528134)                                     -13.946274
## h(-1.14897-ManufacturingProcess09)                                      -1.188263
## h(ManufacturingProcess09- -1.14897)                                      0.928075
## h(ManufacturingProcess32- -1.2965)                                       0.672859
## h(0.0172987-ManufacturingProcess39)                                     -0.347847
## h(ManufacturingProcess01- -0.830301) * h(ManufacturingProcess04-0.0409632)  0.618269
## h(ManufacturingProcess01- -0.830301) * h(-0.245576-ManufacturingProcess33)  0.404746
## h(ManufacturingProcess02-0.504285) * h(ManufacturingProcess39-0.0172987)   51.212532
## h(ManufacturingProcess04- -0.592305) * h(ManufacturingProcess32- -1.2965)   0.735769
## h(ManufacturingProcess04- -0.275671) * h(ManufacturingProcess39-0.0172987) -7.366225
## h(0.273211-ManufacturingProcess06) * h(ManufacturingProcess09- -1.14897)   -0.928072
## h(-0.410521-ManufacturingProcess17) * h(ManufacturingProcess39-0.0172987)   6.894512
##
## Selected 13 of 54 terms, and 9 of 56 predictors (nprune=13)
## Termination condition: GRSq -10 at 54 terms
## Importance: ManufacturingProcess32, ManufacturingProcess09, ...
## Number of terms at each degree of interaction: 1 5 7
## GCV 0.8127773    RSS 72.0719    GRSq 0.7533046    RSq 0.8459556
```

Manufacturing process variables completely dominate the importance list — no biological material variables. ManufacturingProcess32 leads at 100%, followed by ManufacturingProcess09 (71.36%), ManufacturingProcess06 (46.82%), ManufacturingProcess17 and ManufacturingProcess39 (both at 41.93%), and ManufacturingProcess04 (37.31%). ManufacturingProcess01 and ManufacturingProcess33 also contribute at 21.91% each, appearing primarily through interaction terms. The final MARS model selected 13 terms from 9 predictors, including seven interaction terms — for example, ManufacturingProcess02 × ManufacturingProcess39 carries a very large coefficient (51.21), suggesting yield is highly sensitive to the joint behavior of these two process variables. This is something a linear model would miss. Compared to the Chapter 6 linear model, which typically surfaces a mix of both biological and process variables, the nonlinear MARS model points exclusively to process conditions and their interactions as the key drivers of yield.

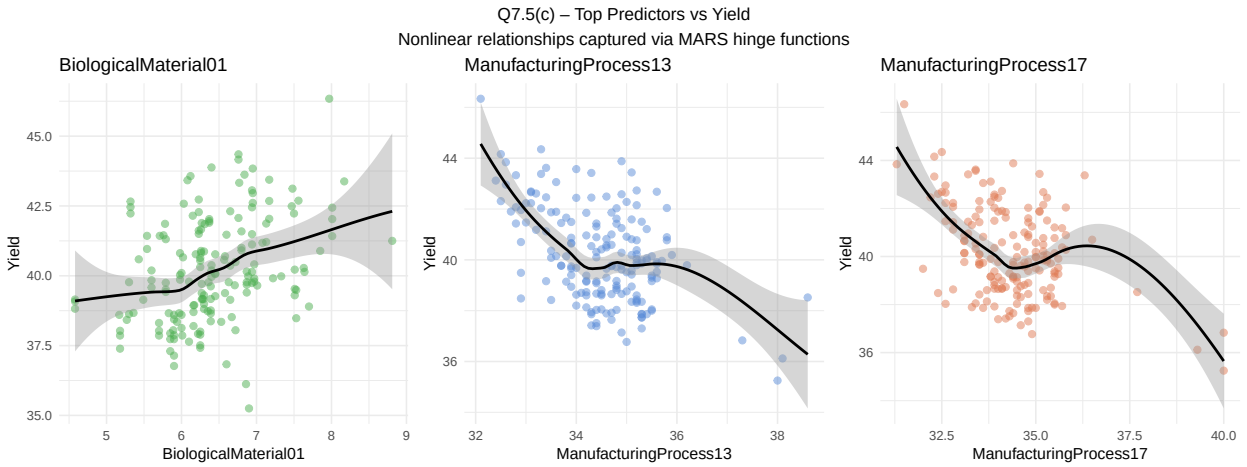
(c) Explore the relationships between the top predictors and the response for the predictors that are unique to the optimal nonlinear regression model. Do these plots reveal intuition about the biological or process predictors and their relationship with yield?

```
top_preds <- c("BiologicalMaterial01",
               "ManufacturingProcess13",
               "ManufacturingProcess17")
colors_c  <- c("#4CAF50", "#5b8dd9", "#e07b54")

plot_list <- lapply(seq_along(top_preds), function(i) {
  pvar   <- top_preds[i]
  df_tmp <- data.frame(
    x = ChemicalManufacturingProcess[[pvar]],
    y = ChemicalManufacturingProcess$Yield
  )
  df_tmp <- df_tmp[!is.na(df_tmp$x), ]
  ggplot(df_tmp, aes(x = x, y = y)) +
    geom_point(alpha = 0.5, color = colors_c[i], size = 2) +
    geom_smooth(method = "loess", se = TRUE,
                color = "black", linewidth = 0.9) +
    labs(title = pvar, x = pvar, y = "Yield") +
    theme_minimal(base_size = 11)
})

grid.arrange(
  grobs = plot_list,
  ncol  = 3,
  top   = "Q7.5(c) - Top Predictors vs Yield\nNonlinear relationships captured via MARS hinge functions
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



The three scatter plots shiw clear nonlinear patterns. BiologicalMaterial01 shows a gradual positive relationship with yield — as the biological input quality increases from roughly 5 to 7, yield rises steadily, then flattens out, suggesting a saturation effect at higher values. ManufacturingProcess13 shows a more complex inverted pattern: yield is highest at lower process values (around 32–34) and drops sharply as the variable increases, indicating there is an optimal operating range beyond which this process setting hurts yield. ManufacturingProcess17 tells a similar story — yield peaks at moderate values (around 34–36) and declines on both ends, pointing to a nonlinear sweet spot. These patterns make physical sense, biological input quality sets a baseline for yield, while process variables have optimal operating windows that, if exceeded, can degrade the outcome. The LOESS smoothers confirm that none of these relationships are purely linear, validating the use of a nonlinear model like MARS over a standard linear regression.